these tests to avoid the additional setup, but I will note that the action tests for creating a project in test/actions/creates_project_test.rb didn't have to be changed.

Restricting Access

Having required a login for your application, you've solved part of the potential security problem. The next problem involves limiting a user's access to projects the user is associated with.

Let's start with an integration test. The test needs as its *given* a project and at least two users—one who has access and one who does not. The *when* action is an attempt to view the project show page, and the *then* specification is the successful or unsuccessful page view. There are a couple of other security aspects you might test, such as whether the index list of projects is filtered by what projects the user is part of, whether a user can edit or create a project, and so on. But this set of tests will give the basic idea:

security/02/spec/system/user_and_role_spec.rb

```
describe "roles" do

  let(:user) { create(:user) }
  let(:project) { create(:project, name: "Project Gutenberg") }

  it "allows a user who is part of a project to see that project" do
    project.roles.create(user: user)
    login_as(user)
    visit(project_path(project))
    expect(current_path).to eq(project_path(project))
  end

  it "doesn't allow a user who is not part of a project to see the project" do
    login_as(user)
    visit(project_path(project))
    expect(current_path).not_to eq(project_path(project))
  end

end
```

The tests both create a project, log in as a user, and visit the project page. In the first test you also add the user to the project. In the second test you assert that the user goes anywhere but the actual project page. Both tests fail at the moment.

In writing these tests, you have a new concept to represent in the code: the combination of a user and a project. This requires you to make a design decision. As you write the test, you're making a claim about how you want the application data to be structured. Often just planning the test will expose the need for new data or new structures, and something as simple as this migration might be done before writing the test. That's fine as long as the larger idea of using the test process to drive the design of the code still holds.